

Adquisición de Datos

APIs, scraping y el desorden del mundo real

Enrique Bolaños Abner Xocop

30 de julio de 2026

Veritas · Libertas · Jvstitia

Agenda

- 1.** ¿De dónde salen los datos?
- 2.** APIs REST
- 3.** Web scraping
- 4.** Datos sucios
- 5.** Ética y procedencia
- 6.** Ejercicios
- 7.** Cierre

El dataset limpio es una ficción escolar

Hasta hoy les hemos dado el CSV listo. En el trabajo real nadie se los da.

Las cuatro fuentes que van a encontrar:

1. **Archivos:** CSV, Excel, Parquet, exportaciones de un sistema
2. **APIs:** un servidor que responde datos estructurados (JSON)
3. **Web scraping:** la página existe, la API no
4. **Sensores y logs:** flujos continuos, enormes y sucios

La cifra que se repite en la industria: entre el 60 % y el 80 % del tiempo de un proyecto se va en conseguir y limpiar los datos, no en modelar.

Conseguir el dato ES parte del trabajo técnico

HTTP en cinco minutos

Todo lo de hoy es la misma conversación: su programa manda un **request**, un servidor devuelve un **response**.

- **URL**: qué recurso quieren
- **Método**: GET para leer, POST para enviar
- **Headers**: metadatos (formato, identidad, autenticación)
- **Body**: el contenido, casi siempre JSON o HTML

Los códigos de estado que sí importan:

200	Todo bien
404	No existe: revisen la URL
429	Van muy rápido: hay que esperar
500	El servidor falló, no ustedes

Primera llamada con requests

Open-Meteo es una API pública y gratuita de clima, sin llave.

```
1 import requests
2 url = "https://api.open-meteo.com/v1/forecast"
3 params = {
4     "latitude": 14.63,
5     "longitude": -90.51,
6     "daily": "temperature_2m_max",
7     "timezone": "America/Guatemala",
8 }
9 r = requests.get(url, params=params, timeout=10)
10 r.status_code      # 200 esperado
11 datos = r.json()   # dict de Python
```

params construye la query string por ustedes, con el escapado correcto.

De JSON anidado a DataFrame

El JSON casi nunca viene plano. `json_normalize` aplanar los niveles.

```
1 import pandas as pd
2
3 clima = pd.DataFrame(datos["daily"])
4 clima["time"] = pd.to_datetime(clima["time"])
5 clima.head()
6 # cuando el JSON viene anidado de verdad:
7 plano = pd.json_normalize(datos, sep="_")
8 plano.columns    # daily_time, daily_temperature...
```

Regla: primero **miren** el JSON (`datos.keys()`), después decidan cómo aplanarlo.

APIs con parámetros: indicadores de un país

El Banco Mundial expone indicadores por país en una API abierta. El patrón de URL es el mismo: recurso más parámetros.

```
1 base = "https://api.worldbank.org/v2"
2 ruta = "/country/GTM/indicator/NY.GDP.PCAP.CD"
3
4 r = requests.get(base + ruta,
5                  params={"format": "json",
6                          "per_page": 100},
7                  timeout=10)
8 meta, filas = r.json() # respuesta de 2 elementos
9 pib = pd.json_normalize(filas)
10 pib[["date", "value"]].head()
```

GTM es Guatemala y NY.GDP.PCAP.CD es PIB per cápita.

Llaves, paginación y límites de uso

```
1 import os, time
2 # La llave NUNCA va escrita en el código
3 llave = os.environ["MI_API_KEY"]
4 headers = {"Authorization": f"Bearer {llave}"}
5 # Paginación: pedir de a poco, acumular
6 todo = []
7 for pagina in range(1, 6):
8     r = requests.get(url, params={"page": pagina},
9                       headers=headers, timeout=10)
10    if r.status_code != 200:
11        break
12    todo.extend(r.json())
13    time.sleep(0.5)    # cortesía
```

Un 429 significa que pasaron el **rate limit**. Reintentar más rápido no ayuda.

Higiene al consumir una API

- Siempre timeout: sin él, su script puede colgarse para siempre
- Siempre revisar `status_code` antes de leer `.json()`
- Guardar la respuesta **cruda** en disco antes de transformarla
- Nunca subir llaves a GitHub: variables de entorno o archivo `.env` ignorado

MÁS ALLÁ DEL CÓDIGO

Guardar el JSON crudo parece redundante hasta el día en que la API cambia, se cae o les corta el acceso. Ese archivo es la única prueba de qué decían los datos cuando los bajaron.

HTML mínimo para scrapear

Una página es un árbol de etiquetas con atributos.

```
1 <table class="wikitable">
2   <tr><th>Departamento</th><th>Poblacion</th></tr>
3   <tr><td>Guatemala</td><td>3015081</td></tr>
4   <tr><td>Huehuetenango</td><td>1170669</td></tr>
5 </table>
```

- **Etiqueta:** table, tr, td, div, a
- **Atributos:** class, id, href
- Scrapear es **navegar ese árbol** y sacar el texto de los nodos correctos

Abran el inspector del navegador (clic derecho, Inspeccionar): ahí encuentran la etiqueta y la clase que necesitan.

BeautifulSoup: find y select

```
1 from bs4 import BeautifulSoup
2
3 html = requests.get(url, timeout=10).text
4 sopa = BeautifulSoup(html, "html.parser")
5
6 sopa.find("h1").text
7 sopa.find("table", class_="wikitable")
8 sopa.find_all("a")[:5]
9
10 # select usa selectores CSS
11 sopa.select("table.wikitable tr")
12 [td.get_text(strip=True)
13  for td in sopa.select("table.wikitable td")][:6]
```

find devuelve el primero, find_all y select devuelven listas.

read_html: el atajo para tablas

Si lo que quieren ya es una tabla HTML, pandas la convierte sola.

```
1 url = ("https://es.wikipedia.org/wiki/"
2        "Departamentos_de_Guatemala")
3
4 cab = {"User-Agent": "curso-ds-ufm/1.0"}
5 resp = requests.get(url, headers=cab, timeout=20)
6 tablas = pd.read_html(io.StringIO(resp.text))
7 len(tablas)
8 df = tablas[0]
```

Con la URL directa, Wikipedia responde 403 al agente por defecto; y pasar el HTML como cadena está deprecado, de ahí `io.StringIO`. Requiere `lxml`. Si la tabla se genera con JavaScript, `read_html` no la ve: ahí hace falta Selenium o Playwright.

Encodings: la ñ que se convierte en basura

Un archivo de texto es bytes; el **encoding** dice cómo leerlos.

```
1 # Sale "Quetzaltenango" mal escrito?
2 df = pd.read_csv("municipios.csv")
3
4 # Muchos sistemas latinoamericanos exportan latin-1
5 df = pd.read_csv("municipios.csv",
6                  encoding="latin-1")
7
8 # Ultimo recurso: no fallar por un byte raro
9 df = pd.read_csv("municipios.csv",
10                  encoding="utf-8",
11                  encoding_errors="replace")
```

Si ven Ã± donde debería haber ñ, leyeron latin-1 como si fuera utf-8.

Fechas, duplicados y centinelas

```
1 # fechas en formatos mezclados
2 df["fecha"] = pd.to_datetime(df["fecha"],
3                               errors="coerce")
4 df["fecha"].isna().sum() # las que no se pudieron
5
6 # duplicados: exactos o por llave
7 df.duplicated().sum()
8 df = df.drop_duplicates(subset=["id"])
9
10 # centinelas disfrazados de dato
11 df = df.replace([-999, "N/A", "", "NULL"], pd.NA)
```

Un -999 promediado arruina el análisis

Cómo se ve un dato sucio de verdad

- Números guardados como texto: "1,250.00", "Q 340"
- Categorías inconsistentes: "Guatemala", "guatemala", "GUATE"
- Espacios invisibles al inicio o al final del texto
- Unidades mezcladas en la misma columna: metros y pies
- Fechas ambiguas: 03/04/2026 puede ser marzo o abril

Herramientas de primera línea: `str.strip()`, `str.lower()`, `str.replace()`, `value_counts()` para ver las categorías reales, y `pd.to_numeric(..., errors="coerce")`.

`value_counts()` sobre cada columna categórica antes de modelar: ese hábito les va a ahorrar horas.

Que se pueda no significa que se deba

Antes de bajar cualquier cosa, tres preguntas:

1. **¿Me lo permiten?** robots.txt y los términos de uso del sitio
2. **¿Hay personas adentro?** Nombres, cédulas, ubicaciones, salud
3. **¿Estoy haciendo daño?** Miles de requests son un ataque, no una consulta

Scraping responsable: User-Agent honesto, time.sleep entre requests, bajar una sola vez, y preferir la API si existe.

MÁS ALLÁ DEL CÓDIGO

Los datos personales siguen siendo personales aunque estén públicos. Que una página sea accesible no otorga consentimiento para agregar, cruzar y republicar la información de alguien.

Procedencia: documentar de dónde salió el dato

Un dataset sin procedencia no es evidencia: es un rumor en formato tabla.

```
1 procedencia = {  
2     "fuente": "Open-Meteo API v1",  
3     "url": url,  
4     "parametros": params,  
5     "fecha_descarga": "2026-07-30",  
6     "filas": len(clima),  
7     "licencia": "CC BY 4.0",  
8 }
```

Guárdenlo junto al archivo crudo. En este curso lo vamos a exigir: toda HDT y todo proyecto que use datos externos debe decir **de dónde salieron y cuándo**.

Sin esa ficha, nadie (ni ustedes en tres meses) puede reproducir el análisis.

Ejercicio 1: una API a DataFrame

```
1 url = "https://api.open-meteo.com/v1/forecast"
2 params = {"latitude": 14.63, "longitude": -90.51,
3           "daily": "temperature_2m_max,",
4                  "precipitation_sum",
5           "timezone": "America/Guatemala"}
6
7 # a) hagan el GET y verifiquen status_code
8 # b) construyan un DataFrame de datos["daily"]
9 # c) conviertan "time" a datetime
10 # d) ¿que día se espera mas lluvia?
11
12 # ¿Que va aquí?
```

Ejercicio 2: scrapear, limpiar y documentar

```
1 url = ("https://es.wikipedia.org/wiki/"
2        "Departamentos_de_Guatemala")
3
4 # a) requests (User-Agent propio) + StringIO +
5 #     read_html; elijan la tabla correcta
6 # b) renombren columnas a nombres cortos y limpios
7 # c) quiten comas y espacios; a numerico con
8 #     pd.to_numeric(..., errors="coerce")
9 # d) ordenen por poblacion y saquen el top 5
10 # e) escriban la ficha de procedencia (fuente,
11 #    url, fecha, filas, licencia) y guardenla
12 #    como JSON junto al CSV limpio
13
14 # ¿Que va aqui?
```

El inciso (e) es el que convierte un script en trabajo defendible.

Lo esencial de hoy

- Conseguir y limpiar los datos **es** el trabajo, no el preámbulo
- HTTP: request, response, y los códigos 200, 404, 429, 500
- `requests + params + json_normalize` para APIs REST
- `read_html` para tablas; BeautifulSoup cuando haga falta precisión
- Encodings, fechas mixtas, duplicados y centinelas: los cuatro sospechosos
- **Procedencia:** de dónde salió el dato y cuándo, siempre documentado

**Próxima clase (martes 4 de agosto):
Probabilidad, MLE y MAP**

Ese día se entrega la **HDT 1** y se asigna la **HDT 2**